

Shaving the Peaks: Taming Tail Latency for Managed Workloads via Disaggregated Garbage Collection

Abstract

Language runtimes are essential systems commonly used in multi-tenant cloud scenarios, such as interactive web services and function-as-a-service (FaaS) applications. They usually provide memory management services, or garbage collections (GC), to automatically reclaim memory and reduce the labor work of application developers. Recent concurrent collectors allow GC to co-run with application threads (mutators), which reduces the application pauses and intends to improve the applications' tail latency. However, this work observes that periodic GC workloads remain a primary source of long tail latency, particularly in resource-constrained multi-tenant environments. In such settings, GC threads consume significant CPU resources, leading to severe performance contention with mutators.

To resolve the contention, this work provides DGC, a disaggregated GC architecture that provides GC as an external service. DGC decouples the most costly marking phase in concurrent GC and offloads it to a disaggregated marking engine. Through a co-design of the GC marking algorithm and an RDMA-based software paging mechanism, DGC's disaggregated marking engine achieves performance on par with local execution while offloading marking to a remote node. To improve resource utilization, DGC introduces a global GC orchestrator to serve multiple runtimes while minimizing the conflicts due to the overlapping of individual GC triggering points. DGC is implemented on OpenJDK HotSpot Java virtual machine, and the evaluation results on representative latency-sensitive applications show that DGC reduces P99 latency by up to 64.4% under moderate workloads and improves the peak goodput by up to 24.0%.

1 Introduction

Language runtimes provide execution environments for high-level programming languages, including OpenJDK HotSpot (Java), V8 (JavaScript), ART (Kotlin), etc. One of the most important features in language runtimes is their automatic

memory management module (namely garbage collection or GC), which helps identify unused memory resources and reclaim them for later reuse. Recent garbage collectors usually embrace a *concurrent* design, allowing GC threads to execute simultaneously with application threads (or *mutators*). Compared with other collectors inducing stop-the-world (STW) pauses, concurrent collectors do not require mutators to pause during GC and are theoretically beneficial to latency-sensitive applications.

However, the advantages of concurrent collectors are severely compromised in modern multi-tenant cloud environments, where applications are typically containerized with strict CPU resource limits to maximize resource utilization and optimize overall infrastructure cost. In such scenarios, the concurrent execution model creates a fundamental conflict: CPU-intensive GC marking threads must compete directly with mutators for the CPU resources. Our analysis confirms that this resource contention causes application-side CPU starvation. For instance, under the SPECjbb2015 benchmark, Shenandoah (a representative concurrent collector in OpenJDK) introduces a severe latency spike when GC marking is active: as the application's available CPU share drops to 60% of its normal demand, its average latency rises by more than an order of magnitude. However, once GC threads become inactive, the CPU becomes underutilized under the same workload.

To address the issues of resource utilization fluctuations and tail latency caused by periodic GC tasks, this work proposes a novel architecture that decouples GC tasks from the original runtime and abstracts them into a dedicated service. This approach significantly reduces interference with mutators in the original runtimes (referred to as *served runtimes*). Additionally, the service can be time-shared by multiple runtimes, improving the overall system resource utilization. Thanks to recent high-performance interconnection technologies, the service can also be initiated remotely and serve language runtimes running on different physical machines. Inspired by the concept of resource pooling and disaggregation, we name this architecture as *disaggregated GC* (DGC).

The design of DGC mainly faces two challenges:

How to achieve correct and efficient remote marking?

Cross-machine concurrent marking inherently introduces consistency and data plane overheads typical of distributed systems. To address these challenges, DGC proposes a *disaggregated marking engine*. It first employs a lightweight snapshot-at-the-beginning (SATB) protocol, which minimizes the synchronization overhead between GC markers and application threads. Second, to address data plane inefficiencies, it introduces a co-design of the marking logic and the paging subsystems. DGC utilizes the runtime’s coarse-grained memory management unit (namely *regions*) to construct a software-based RDMA paging system, which reduces the address translation cost associated with software page tables. Furthermore, DGC proposes a marking algorithm to operate on incomplete object graphs, which ensures overlapping between object marking and RDMA-based data retrieving and thus achieves marking performance comparable to local execution.

How to serve multiple runtime instances simultaneously? Existing language runtimes schedule GC assuming it monopolizes the computation resources. However, the assumption does not hold with DGC, where the computation resources are shared by multiple runtimes, and relying on the original scheduling leads to suboptimal results. To address this, DGC employs lightweight memory allocation monitors embedded within each runtime to formulate a forward-looking DGC orchestration plan. This design enables DGC to fully utilize the marking offload capabilities to accelerate participating runtimes, all while avoiding contention between concurrent DGC tasks.

DGC is implemented on OpenJDK HotSpot, a widely used JVM. It is based atop Shenandoah, a concurrent garbage collector whose SATB marking design is representative and can be easily migrated to other collectors [7, 50]. The evaluation results on various latency-sensitive applications show that DGC can achieve significant (up to 64.4%) P99 latency reduction against two state-of-the-art collectors with identical CPU resource budgets. Despite the network overhead, its remote version can reach 60.3% P99 latency reduction under moderate workloads.

To summarize, the contribution of this work includes the following:

- A comprehensive analysis to illustrate the periodic resource contention between GC and application threads, especially for multi-tenant cloud scenarios. (§ 3)
- DGC, a disaggregated GC architecture to elastically serve GC tasks in an external instance to reduce GC-mutator conflicts. (§ 4-6)
- Experiments on various latency-critical workloads to show how DGC improves applications’ tail latency (§ 8).

2 Background

Garbage Collection. Garbage Collection (GC) automates memory management by dynamically tracking object liveness and reclaiming unused resources. This mechanism relieves developers from low-level manual administration and prevents memory-related bugs, such as leaks, which are prevalent in complex software systems.

The mainstream GC algorithm is *tracing*, which identifies live objects by traversing the object graph and marking all reachable ones. During marking, GC threads start with root objects (referred to as *GC roots*) known to be alive (e.g., used by on-stack references) and traverse the heap (or a part of the heap) to locate live objects. After marking, *non-moving* collectors reclaim areas consumed by dead objects and organize them into free lists, while *moving* collectors copy live objects to adjacent locations (known as *evacuation*) to create a large and contiguous free space.

Concurrent Marking. Performance overhead is a key concern of GC. Owing to the consistency issues between GC threads and application threads (referred to as *mutators*), early garbage collectors relied on stop-the-world (STW) pauses, suspending all application threads during object marking, which significantly affects the application latency. In contrast, modern garbage collectors employ advanced techniques such as barriers and sophisticated algorithmic designs, enabling concurrent execution with mutators, especially for the marking phase (namely *concurrent marking*). Nowadays, many language runtimes (OpenJDK HotSpot [34], V8 [12], Go runtime [11], ART [10], etc.) have shifted towards concurrent marking. The reduced pause times achieved through concurrent marking are particularly crucial for latency-critical applications, such as databases and web services, which are mostly written in high-level languages [14].

Marking Algorithm. Marking algorithms rely on the tri-color abstraction to reason about graph traversal correctness. Objects are categorized into three states:

- *White*: Unvisited candidates. Any object remaining white upon termination is unreachable and eligible for reclamation.
- *Grey*: Live objects that have been discovered but not yet scanned for outgoing references. These reside in the collector’s work buffers (e.g., queues) and represent the active wavefront of the traversal.
- *Black*: Fully processed objects whose direct children have all been identified.

The marking process proceeds iteratively: the collector pops a grey object, marks its children grey, and colors the object black. To guarantee convergence and idempotency, modern collectors (e.g., Shenandoah) utilize a global mark bitmap. This atomic side-table ensures that each reachable

Table 1: Results of SPECjbb2015 for different GCs

	Shenandoah	ZGC	G1
(A): Critical-jOPS (req/s) vs. GC Threads			
2 Threads	4937	3481	2530
4 Threads	6710	5512	3745
6 Threads	6569	6262	5208
8 Threads	6819	6474	6288
(B): Pause Metrics (under Critical-jOPS, 8 Threads)			
Pause Count	45	71	49
Avg Pause (ms)	1.91	0.04	51.44
Total Pause (ms)	85.79	3.20	2520.59

object transitions from white to grey exactly once, preventing redundant processing of cycles or shared references.

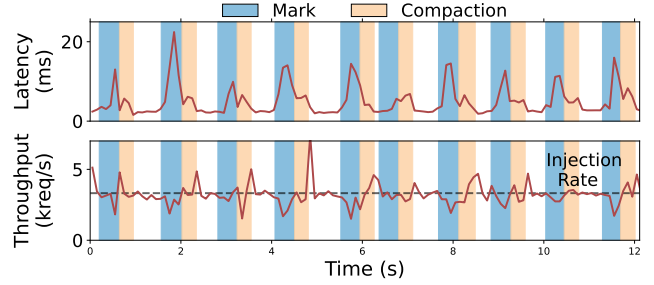
Microservices and Multi-tenancy. Microservices have become a prevailing architectural choice for cloud-native applications, replacing traditional monoliths to support elastic scaling. Unlike traditional monoliths that rely on vertical scaling (scale-up) of massive, singular instances, modern hyperscale applications are decomposed into thousands of loosely coupled, fine-grained service units. This architectural decomposition is primarily driven by the need for horizontal scalability (scale-out). By breaking down complex logic into smaller components, operators can dynamically provision or terminate replicas of standardized service instances to match real-time traffic fluctuations, ensuring agility and fault tolerance.

Consequently, the deployment strategy has shifted towards co-locating high densities of these smaller service instances to maximize cluster-wide resource utilization. To achieve efficient bin-packing, these instances are typically constrained to moderate specifications (According to a recent report, 82.8% of monitored JVMs use between 1 and 8 cores, and 76.2% are allocated less than 4 GB of memory [32].) While this model optimizes infrastructure costs, it fundamentally constrains language runtimes’ execution environment: the surplus CPU cycles previously relied upon for background maintenance tasks are largely eliminated, creating contention between application logic and runtime services like GC.

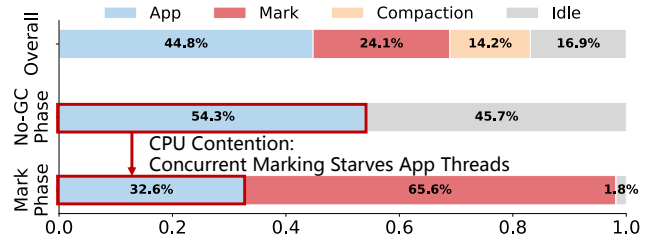
3 Motivation

In this section, we use a set of experiments to quantitatively demonstrate (1) how concurrent marking competes with application threads for CPU resources in resource-constrained environments and leads to performance fluctuations, and (2) why existing configuration adjustments cannot effectively solve these issues.

Experiment Setup. We conducted our experiments using SPECjbb2015 (abbreviated as SPECjbb), a benchmark representing typical online shopping workloads, across three GCs



(a) SPECjbb2015 latency and throughput details. Note that SPECjbb2015 employs a queue for unhandled requests, which leads to rebounds in throughput after CPU contention finishes.



(b) CPU utilization breakdown during different GC stages.

Figure 1: Performance degradation caused by concurrent GC.

in OpenJDK 17. We provisioned the SPECjbb instance with 8 cores and 4 GB of RAM to reflect a resource-limited, multi-tenant environment. Notably, the 4 GB limit provides twice the minimum memory required by the workload, guaranteeing adequate space for allocation and garbage collection. SPECjbb incorporates a built-in load-probing mechanism that automatically evaluates latency characteristics under varying loads and reports the maximum throughput satisfying its latency constraints (referred to as critical-jOPS).

Three collectors are selected for evaluation. *G1* serves as the mostly-STW baseline, utilizing a generational design with frequent pauses for young-generation reclamation. In contrast, *Shenandoah* and *ZGC* represent the concurrent category; they leverage sophisticated consistency protocols to achieve a ‘pause-less’ execution model, confining STW interruptions strictly to sub-millisecond phase transitions.

Performance Analysis. The results presented in Table 1 reveal a counterintuitive phenomenon: although concurrent collectors succeed in reducing STW pauses, they yield only marginal improvements in critical-jOPS (3-8%) compared to G1. To investigate the root cause of this phenomenon, we conducted a detailed performance analysis of Shenandoah (selected over ZGC due to its lighter-weight concurrent algorithm and better critical-jOPS) under the IR=6819 configuration. In order to get the performance details, we instrumented SPECjbb to collect fine-grained performance metrics during testing, correlating them with system resource utilization statistics (summarized in Figure 1).

The results uncover a critical pattern: during non-GC periods, CPU utilization remains consistently low (45.7% Idle), indicating CPU underutilization. However, during concurrent marking, total CPU usage spikes to 98.2% as the application’s CPU share drops from 53.3% to 32.6%, resulting in a sharp drop in application throughput (47%) and increased average latency (10.5×). This observation suggests an explanation for the limited critical-jOPS improvement: while traditional STW collectors completely hand over CPU resources to either application or GC threads at different phases, concurrent GCs periodically introduce heavy GC tasks colocated with mutators. Although pauses have been reduced, those GC tasks still interfere with application threads, inducing recurring resource contention. The resource contention prevents concurrent GCs from achieving truly stable performance, undermining their theoretical advantage.

Why not overprovisioning? The most straightforward solution to this issue is CPU overprovisioning—allocating sufficient cores to accommodate peak demand from both application and GC threads. This approach would prevent performance fluctuations caused by CPU contention when GC activates, as incidentally observed in our Figure 1 experiment, where the system automatically settled at 6,819 req/s, a moderate load level that balanced jOPS and P99 latency. However, while CPU overprovisioning is simple and effective, it introduces significant efficiency concerns. As demonstrated in our earlier results, this method leads to severe underutilization during non-GC periods, with CPU utilization dropping below 50%—a substantial resource waste.

An alternative overprovisioning solution is to grant extra memory to the application. For garbage-collected applications, expanding memory quotas can reduce GC frequency and consequently mitigate its performance impact. However, this approach also has its limitations. The CPU-to-memory ratio of current machines is essentially fixed in existing public clouds or private data centers, where memory capacity is already the primary bottleneck during resource allocation[14, 39]. Therefore, adding more idle memory to the application only for GC frequency reduction is not an effective solution.

Why not reduce GC Threads? GC Thread number is one of the most important parameters for GC performance tuning. Given that the root cause of CPU load fluctuations lies in the periodic activation of GC tasks, one possible solution is to reduce GC threads to stretch GC execution time, thereby smoothing overall system load. Table 2 presents performance data from SPECjbb tests under four different GC thread configurations.

Reducing GC threads from 8 to 6 and 4 initially demonstrates a decline in GC throughput (defined as live object size divided by marking time), confirming the ability to stretch GC duration through this method. However, the system exhibits an 11% increase in GC frequency (8 to 4), while P99 latency remains statistically unchanged and P50 latency even

Table 2: Tail latency with different numbers of marker threads. Tested on SPECjbb2015 (PRESET mode, IR=6819 req/s).

GC Metrics (marking)				App Latency (ms $\pm$ 95% CI)		
Thr.	#GC	GB/s	CPU%	P50	P90	P99
2T	41	1.04	24.5%	1650 _(oom)	3600 _(oom)	11500 _(oom)
4T	50	1.98	46.2%	4.8 $\pm$ 0.1	19.3 $\pm$ 3.9	76.7 $\pm$ 9.5
6T	46	2.48	54.6%	4.2 $\pm$ 0.1	26.6 $\pm$ 2.3	88.8 $\pm$ 16.3
8T	45	2.57	55.4%	4.0 $\pm$ 0.1	21.4 $\pm$ 1.4	78.3 $\pm$ 18.9

increases by 20%. This result is attributed to the *floating garbage* problem: modern concurrent collectors (e.g., Shenandoah and ZGC) conservatively treat objects dying mid-cycle as live to ensure correctness, limiting reclamation to objects already dead at cycle start. Consequently, while fewer GC threads reduce CPU occupancy, the GC duration is longer due to less GC throughput, which requires earlier GC triggers to avoid out-of-memory (OOM) issues and thus amplifies the floating garbage problem. This undermines per-cycle reclamation efficiency, elevates collection frequency, and ultimately introduces non-trivial systemic overhead. When decreasing the GC thread number to 2, the GC throughput falls behind the allocation rate, triggering repeated OOM and causing stop-the-world GC fallback, leading to severe performance degradation.

Opportunities. While local tuning falls short, modern cloud infrastructure presents two key opportunities to resolve GC-induced contention: (1) *Utilizing Idle Clusters Cycles*. In multi-tenant environments, GC bursts are rarely synchronized across all co-located runtimes. While a single runtime struggles with CPU deficits during its GC phase, neighboring runtimes may possess underutilized CPU cycles. This asymmetry creates an opportunity to aggregate these fragmented, idle resources into a shared pool. By offloading GC tasks to this isolated pool, we can achieve agile concurrent marking without contending with local application threads. (2) *The Bandwidth-Compute Gap*. The emergence of high-performance interconnects (e.g., RDMA, CXL) has fundamentally shifted the bottleneck. As shown in Table 2, concurrent marking is often bound by random memory access latency and synchronization overheads, yielding a modest scan speed of 2.6 GB/s in our tests. In contrast, modern 400 Gbps RDMA networks offer bandwidth orders of magnitude higher—theoretically sufficient to sustain traffic for 20 different marking tasks simultaneously. This vast bandwidth headroom makes it feasible to decouple GC to remote nodes, trading abundant network bandwidth for scarce local CPU resources.

Takeway. Concurrent garbage collectors in multi-tenant environments compromise application stability primarily through CPU contention. Viewing the runtime holistically, this contention manifests as drastic, periodic fluctuations in CPU

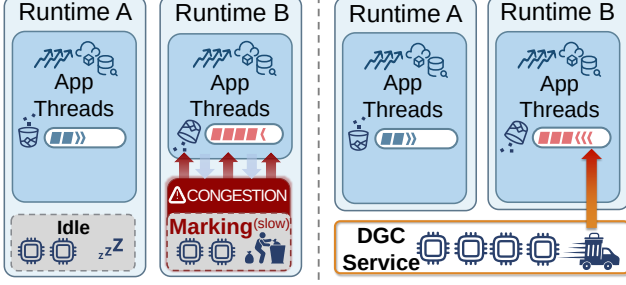


Figure 2: Overview of the baseline (left) and DGC (right).

demand, making precise resource provisioning notoriously difficult. Consequently, the theoretically optimal solution requires elasticity: the runtime can access burstable CPU resources during GC phases (marking in particular) to reduce floating garbage, yet these resources should not be statically provisioned to avoid wastage during non-GC phases. This necessitates a consolidated approach: offloading GC tasks from multiple runtimes onto a shared, dedicated resource pool, where GC tasks are explicitly orchestrated to ensure efficient, interference-free execution.

4 Disaggregated GC Service

This work presents Disaggregated Garbage Collection (DGC), a system designed to consolidate concurrent marking tasks from multiple runtimes onto a dedicated, isolated compute pool. By decoupling the marking workload from the application, DGC eliminates contention on CPU resources and accelerates marking throughput (Figure 2). The architecture comprises two core subsystems: a Disaggregated Marking Engine and a Global GC Orchestrator.

A strawman design for pooling would be adaptively increasing/decreasing the number of available cores for each language runtime, but this design faces two disadvantages. First, it contradicts the horizontal scaling mechanism used for fine-grained services and complicates resource management. Second, it cannot scale to a distributed environment. DGC instead employs a data-centric offloading strategy: its *disaggregated marking engine* accesses the runtime’s heap through two distinct data planes for concurrent marking.

(1) *Co-located Mode*: For runtimes residing on the same physical host, the DGC service utilizes shared memory (SHM) as the data plane. By registering heap ranges and GC-related data structures into a memory segment visible only to DGC, the DGC service can directly access the runtime’s state. This architecture enables the direct adaptation of standard concurrent marking algorithms, where intermediate metadata is exchanged via zero-copy SHM mechanisms and control signals are managed through simple RPCs. Notably, while currently implemented for single-host co-location, this SHM-

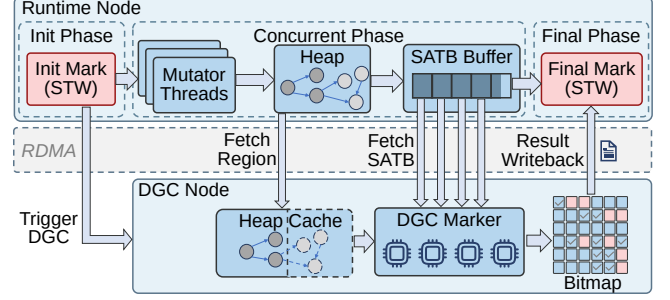


Figure 3: The workflow of DGC’s disaggregated marking.

based design conceptually extends to Hyper-Node systems providing shared memory semantics (e.g. CXL [27], UB [26]).

(2) *Distributed Mode*: For runtimes across physical boundaries, DGC employs RDMA-based Remote Marking, leveraging high-performance networking to traverse remote heaps efficiently (details in Section 5).

To serve multiple participating runtimes using a shared resource pool, the DGC system employs a *Global GC Orchestrator*. This component is composed of lightweight monitors embedded in each runtime and a centralized optimization solver. Using allocation-related profiles collected by monitors, the solver optimizes GC resource scheduling to ensure memory availability of all participating runtimes (details in Section 6).

5 RDMA-based Remote Marking

5.1 SATB-based Marking Algorithm

To address consistency maintenance in disaggregated marking, the remote marking engine adopts the Snapshot-At-The-Beginning (SATB) algorithm. SATB ensures correctness by effectively taking a logical snapshot of the heap at the start of the cycle. Any references overwritten during the concurrent marking phase are recorded by SATB write barriers to ensure that objects live at the start of the cycle remain reachable. Prior work [28, 45] has shown that SATB barriers can ensure the correctness of marking even when GC threads are operating on a stale object graph.

Figure 3 illustrates the workflow of the DGC’s remote marking algorithm. The cycle begins with a lightweight Stop-The-World (STW) phase, during which the runtime records heap states and the GC root set to establish the logical SATB snapshot. Upon the resumption of the application, the DGC service receives an RPC to start data copying and concurrent marking. DGC adopts a configurable heap cache ratio on the GC node, which allows a tradeoff between marking performance and memory consumption. Throughout the marking phase, the accumulated SATB roots are periodically fetched from the runtime node and processed by the GC marker. Once

the scanning converges, the liveness bitmap is returned to the runtime, which then enters a final, brief STW pause to process the small number of remaining SATB roots that were not yet synchronized, ensuring strictly consistent marking results.

5.2 Heap Cache Management

The heap cache manages a (partial) local copy of the served runtime’s memory states, which can be directly accessed by GC threads of DGC. The cache originally supports two primitives: *retrieving* to fetch more memory states from the served runtime, and *eviction* to replace existing cached states with newly retrieved ones. A RDMA control thread is responsible for managing the heap cache, issuing RDMA read requests according to the primitives. A strawman design would be relying on existing OS-level remote memory paging systems [1, 38] to support those two primitives, but this design is intrusive to operating systems and remains unaware of the memory management behaviors inside language runtime systems, leading to frequent page faults and unacceptable overhead. To this end, DGC adopts a user-space software paging system and manages cached memory in a *region* granularity, which aligns with that in runtimes. The software paging approach also reduces the overhead of RDMA registration: during initialization, DGC pre-registers a fixed memory range as a local heap cache and employs a software address translation layer operating at the granularity of regions.

Region-based software paging. The *region* serves as the fundamental unit of memory management in mainstream collectors [7, 8, 53]. By aligning the cache management granularity with regions, DGC offers two advantages. First, it aligns seamlessly with the runtime’s allocation patterns. Specifically, mainstream collectors like G1 and Shenandoah enable a per-region bump-pointer-style allocator, which induces strong temporal locality by grouping related objects to the same region. The locality is also analyzed by prior work, which indicates the number of inter-region references is usually limited [50]. Consequently, retrieving the whole region can improve the “hit rate” of local memory accesses compared with a page-based approach. Second, this approach is highly cache-efficient. For instance, in a 4GB Specjbb heap with 2MB default region size, the software page table shrinks to merely 4KB (properly compressed based on alignment). This compact footprint allows the table to fit entirely within a modern CPU’s L1d cache (48KB in our testbed), significantly reducing address translation overhead.

5.3 Handling Partial Object Graphs

Since the heap cache is empty at the beginning of marking, naively waiting for data copying would introduce significant overhead. Meanwhile, the size of heap cache may not accommodate the whole heap in the runtime node. To this end, DGC refines the original SATB-based GC algorithm to conduct

marking when the cached memory states are not complete. The basic idea is to memorize pending references for uncached regions. During marking, DGC’s GC threads identify those references by checking the status of regions and push them into a thread-local per-region pending queue. Once the region has been copied, DGC uses the RDMA control thread to aggregate pending queues for this region and dispatch them to a single GC thread, according to the current load. This design is aimed at preventing *bitmap contention*: concurrent scanning of the same region by multiple GC threads would trigger frequent cache coherence traffic on the shared marking bitmap.

Concurrency control. The switch point on the region state should be specially designed to ensure consistency between GC threads and the RDMA control thread. For example, if GC threads do not realize that a region is being evicted, they may introduce wrong marking results after the region has been replaced by RDMA read operations issued by the control thread. To this end, DGC introduces an 8-byte per-region entry for inter-thread synchronization: the high bits store the *state tags* (NotCached, OnTrans, Cached, and OnEvict) while the low bits store a counter serving as a reentrant read lock. When accessing an object, a GC thread first issues an atomic increment to the region entry to register its presence, and then validates whether the state tag is *CACHED*. Similarly, it decrements the counter when finish processing the object. The RDMA control thread only changes the state tags when the corresponding counter is zero and leverages compare-and-swap operations to ensure the atomicity of its mutation operation. This concurrency control mechanism allows GC threads to safely initiate tasks with simple atomic instructions while ensuring that critical RDMA overwrites never interrupt an ongoing object scanning.

5.4 Hotness-driven Paging Policies

To minimize RDMA traffic and avoid frequent retrieving on the same region, DGC uses *hotness scores* to determine which region should be retrieved and evicted. The score is calculated by the aggregate size of underprocessed objects (grey) within each region, which is grounded in our observation that larger objects typically encapsulate a higher density of outgoing references. Therefore, caching them exposes more work to GC threads, sustaining high throughput and preventing thread starvation. For regions not cached, DGC initially estimates the score using the number of pending references and the application’s average object size, which can be profiled during each collection cycle. The score can be calibrated to the precise value once the region is retrieved. Furthermore, to prevent thrashing where regions are repeatedly swapped, DGC remains conservative on eviction and enforces a strict hysteresis threshold: A non-resident region is eligible for retrieval only if its score exceeds that of the eviction candidate by a configurable multiplicative factor.

Additionally, to mitigate thread starvation during the initial-

ization phase, DGC introduces a *prefetch window*. This mechanism allows the RDMA controller to proactively populate the local buffer before marking threads start, strictly prioritizing those containing GC roots. Crucially, marking threads remain dormant during this interval. From the perspective of the global GC orchestrator, no CPU resources are allocated for the duration of the prefetch window, thereby preventing idle cycles and maximizing overall resource utilization.

5.5 Class Information Sharing

In addition to heap data, class information is also required during marking, as it stores the locations of all references inside an object. To minimize the remote access to class information, DGC embraces OpenJDK’s application class data sharing mechanism (APPCDS [33]), which packs the information of loaded classes into an archive file. When a served runtime first offloads its marking task to a remote machine, DGC preloads this archive to retrieve class information. For newly loaded classes in the served runtime, we hijack its class loading process to synchronize class-related data structures to DGC.

6 Multi-Runtime GC Orchestration

Although the aforementioned approach establishes an efficient disaggregated GC service, a practical system requires not only the underlying mechanisms but also an appropriate scheduling algorithm serving as the controller. This issue is particularly critical in concurrent GC, where the triggering points are vital to GC performance [19]. Since SATB marking treats all newly allocated objects as alive, initiating it too early can reduce the number of reclaimable objects, leading to suboptimal GC efficiency. On the other hand, triggering GC too late can lead to out-of-memory (OOM) issues and application stalls.

The challenge becomes more severe in DGC as it is shared by multiple runtimes. State-of-the-art collectors already introduce statistical methods to determine the trigger points of concurrent collections. However, those methods assume all computation resources are monopolized by the language runtime and thus perform poorly in DGC’s resource-sharing configuration. To this end, DGC introduces a global resource scheduler that coordinates the GC behaviors of multiple runtimes. By monitoring allocation behaviors across served runtimes and tracking DGC’s resource usage, the scheduler enables efficient and reliable GC task scheduling.

6.1 Analysis and Design Principles

To build a practical GC scheduler that simultaneously serves multiple runtimes, we first introduce our design principles.

Deferring marking as late as possible. As analyzed before, if the trigger point of SATB marking can be deferred, more objects can be included in the snapshot, resulting in less floating garbage and improved GC efficiency. Therefore, DGC’s

scheduler is designed to defer marking for all GC tasks without encountering any OOM issues.

Adjustable GC thread numbers. The number of GC threads is statically configured in state-of-the-art concurrent collectors to strike a balance between collection throughput and interference with mutators. In DGC’s case, since the disaggregated GC tasks are running with dedicated computation resources, the static configuration approach is no longer viable. DGC thereby proposes to dynamically configure the number of GC threads for each task, according to its urgency and the system’s resource availability. When a GC task starts late and requires finishing soon to avoid OOM issues, DGC can maximize its computation resources to improve the marking speed. However, since the SATB marking algorithm (graph traversal) does not scale well (see Table 2), the marginal benefit of adding more threads diminishes beyond a certain threshold. Therefore, when multiple GC tasks require serving simultaneously, the computation resources granted for each should be controlled to improve the overall CPU efficiency.

Avoiding Resource Contention. Since the memory footprint of marking is large, co-locating multiple GC threads into the same core leads to severe interference. To this end, the total number of active GC threads should not exceed the available core number at any moment.

6.2 Problem Formulation

According to the principles above, we formulate the scheduling algorithm as a constrained optimization problem below.

Decision Space: (1) *GC Trigger Time*: The time when the GC task should be triggered. (2) *DGC Thread Number*: The number of GC threads to be allocated for the task.

Constraints: (1) *Time Constraint*: Each GC should complete before the mutators exhaust all the available memory (i.e., OOM errors). (2) *Resource Constraint*: The total number of active GC threads should not exceed the number of available cores at any moment.

Algorithm Input: (1) *GC Completion Deadline*: The estimated GC completion deadline for each served runtime, calculated as the available free memory divided by its allocation rate. (2) *Estimated DGC Duration*: The estimated marking duration of each task under DGC, with different number of GC threads. (3) *Ongoing DGC Task Information*: The information of all ongoing GC tasks served by DGC, including the estimated completion time and the number of GC threads.

Optimization Goal: Defer the GC start time as long as possible. Since the start time can be represented as a monotone timestamp, the goal can be transformed to the sum of the GC start timestamps for all tasks.

Discussion: the estimated inputs. The input of algorithms relies on several estimated numbers, which are collected from runtime profiles in the served runtimes. The allocation rate has

already been estimated by the runtime’s statistical methods and thus can be directly reused by DGC. As for the estimated GC duration, it can be modeled as a linear function of the size of live objects. The function incorporates a thread-number-specific coefficient and a constant term, both of which vary across applications. Given that an application’s live object set typically remains stable over short time periods, the number of live objects can be estimated by the historical marking results.

6.3 Efficient Solving Strategies

To address this optimization problem, we first formulated it as a mixed-integer linear programming (MILP) problem. However, the results were unsatisfactory: despite employing mathematical techniques to linearize all variables and constraints, the excessive number of conditions and auxiliary variables led to poor solver performance. The solver took 700ms to resolve a 4-application problem, which is unacceptable given that a typical GC cycle lasts only a few hundred milliseconds. Moreover, due to the absence of an optimal substructure, the problem could not be reduced to a dynamic programming problem.

Instead, we adopted a constraint programming (CP) approach, discretizing all parameters and integrating a dedicated CP-SAT solver. This reduced the solving time to under 5ms, within the practical limits, considering the allocation rate tracker provided by the runtime operates at 10ms granularity. To handle rare corner cases where the CP model yields no feasible solution, we implemented a fallback mechanism that reverts to a purely local concurrent GC (the same as that in the served runtime).

7 Implementation

DGC is mainly implemented atop the Shenandoah garbage collector of OpenJDK 17 (version: jdk-17+35) in C/C++ and can support any unmodified Java applications. Our implementation has 10,383 LOC, including 946 lines specifically for the local version (referred to as *DGC-SHM*), 3,601 lines specifically for the remote version (referred to as *DGC-RDMA*), 959 lines for the GC scheduler, and the rest are for the common parts (e.g., class data sharing, GC workflow integration, RPC and RDMA support, etc.).

DGC’s disaggregated GC service is implemented as a stripped-down version of OpenJDK’s Java virtual machine (JVM). This enables DGC to reuse critical JVM components, including APPCDS and the class information resolution mechanism. The low-frequency control flow between the disaggregated marker instance and the served JVM is implemented with RPC, while the data flow is implemented with SHM or RDMA. To reach ideal marking performance, DGC reuses the well-optimized data structures in Shenandoah GC, including its work-stealing task queue, lock-free marking bitmaps, etc.

The GC scheduler is implemented with the CP-SAT solver [36] provided by Google OR-Tools [37]. The CP-SAT solver achieves high-performance solving by automatically reducing Constraint Programming (CP) problems to Boolean Satisfiability Problems (SAT). This capability enables its use for real-time GC scheduling in language runtime scenarios.

8 Evaluation

8.1 Experiment Setup

We ran experiments in a cluster of 2 servers with dual Intel Xeon Gold 6430 CPUs (32 physical cores each) and 128 GB of memory. Hyper-threading was disabled, and all servers were configured to performance mode to prevent CPU frequency fluctuation. Each server has a 200Gbps Nvidia BlueField-3 DPU working as an RDMA NIC, which is connected to the host via PCIe 4.0×16 connection.

Test Applications. We leverage the following applications to evaluate DGC:

- *SPECjbb2015* [6] serves as the de facto standard for assessing Java server performance, particularly in relation to garbage collection [8]. It simulates an online supermarket handling diverse requests, and the evaluated version is 1.0.4. SPECjbb2015 supports a multi-backend deployment mode, which natively enables distributed deployment with multiple JVMs. In our evaluation, we utilize this multi-backend mode to simulate a real-world scenario.

- *HBase* [2] is a distributed database designed for non-relational big data storage. The evaluated HBase version is 2.5.11. We use YCSB’s two different mixed workloads (<50% read, 50% update> and <50% read, 50% insert>) [5] to evaluate the performance of DGC in HBase. HBase also supports a native distributed mode, which partitions the data into multiple regions and distributes them across multiple JVMs. In our evaluation, we utilize this multi-regionServer mode to evaluate the performance of DGC.

- *H2* [16] is a relational database written in Java, capable of being embedded within Java applications or operated as a standalone database server. The evaluated H2 version is 2.2.220. To evaluate it under different levels of workloads, we refer to Jade [50]’s h2-throttle test suite [49] to measure the tail latency. This test suite uses TPC-C [43] as the workload, but adds a throttle limit to control the request rate, allowing us to evaluate the performance of DGC under different levels of workloads.

- *DayTrader* [17] is an IBM’s Open Source benchmark application emulating an online stock trading system. We use DaCapo Benchmark [3] tradesoap workload to evaluate the performance of DGC in DayTrader.

Baseline GCs. Our evaluation mainly compares DGC (both DGC-SHM and DGC-RDMA) with Shenandoah and G1. Shenandoah is a concurrent collector integrated in OpenJDK

17. It serves as the codebase for DGC and thus has many implementation similarities. As shown in Table 1, Shenandoah also demonstrates the highest critical-jOPS performance in SPECjbb2015. Therefore, we selected Shenandoah as a representative state-of-the-art concurrent garbage collector to clearly illustrate the performance benefits of DGC.

G1 is currently the default garbage collector in JDK 17. It is a generational collector that is largely stop-the-world (STW): both young-generation collections and the compaction phase during full-heap GC (mixed GC) require STW pauses. Only the marking phase during mixed GC is designed to be concurrent. Although this design leads to frequent application pauses, it results in higher garbage collection efficiency. Due to its widespread adoption in various production scenarios, we used G1 as another baseline.

CPU configurations. To ensure a fair comparison between baseline collectors and DGC, the total CPU resource should be configured to an identical value. For DGC, each served JVM is allocated 8 cores to estimate the resource provisioning in a multi-tenant environment, while the DGC instance gets 2 cores per participating JVM. In contrast, baseline JVMs running native GCs are configured with 10 cores, so the overall core number remains the same. The workloads under test are configured with enough worker threads to fully leverage the 10 cores, whether for application logic or GC tasks.

Memory configurations. For all applications, we first evaluate the minimum heap size that can run them with Shenandoah, then set the evaluation JVM heap size to be 2 $\times$ of the minimum, to simulate a medium workload. DGC-SHM incurs negligible additional memory overhead and can be deployed directly. Conversely, DGC-RDMA relies on a local heap cache to store object graphs. In our primary evaluation, we assume sufficient remote memory availability and thus do not constrain DGC-RDMA’s memory usage. We evaluate the impact of stricter memory configurations separately in Section 8.3, demonstrating that capping memory usage does not significantly compromise DGC-RDMA’s performance gains.

8.2 Macro Benchmarks

In this section, we demonstrate the end-to-end performance improvements of DGC using macro-benchmarks. Since DGC enhances application performance by time-multiplexing CPU resources across instances to mitigate interference, we establish a minimal experimental setup: each application is deployed with two instances, paired with a single 4-core DGC service.

SPECjbb2015. We first evaluate the performance of DGC on SPECjbb2015 with its default mode (HBIR_RT), which measures the latency under different issue rates, calculates the critical-jOPS (maximum achievable issue rate within an SLO limit) and reports a curve. The reported throughput and latency are the summary of the two backend clusters, which

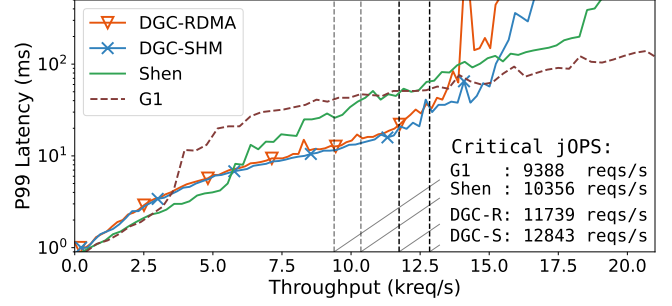


Figure 4: SPECjbb2015’s P99 latency curve and Critical jOPS

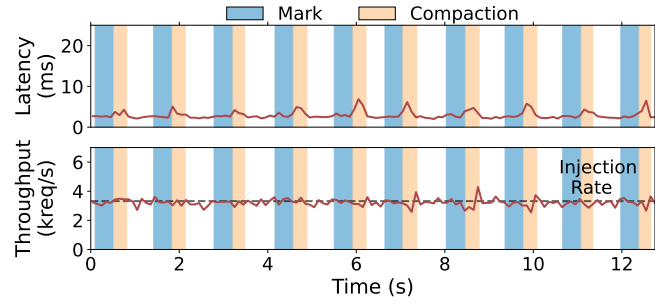


Figure 5: Latency details of SPECjbb2015 with DGC-SHM

leads to a difference with previous results in Section 2.

As shown in Figure 4, DGC-SHM achieves the highest critical-jOPS, with 24.0% and 36.8% higher than Shenandoah and G1, respectively. The DGC-RDMA achieves the second highest critical-jOPS, 8.6% lower than DGC-SHM, but 13.4% higher than Shenandoah and 25.0% higher than G1.

From the P99 curve in Figure 4, it can be observed that G1 exhibits higher latency than other low-pause collectors due to its STW pauses. In contrast, both DGC-SHM and DGC-RDMA maintain lower P99 latency than Shenandoah when the issue rate is between 6k and 13k. In this range, the baseline Shenandoah suffers from high CPU contention between mutator and marker threads, whereas DGC mitigates this by utilizing dedicated physical cores for markers. Consequently, at Shenandoah’s critical jOPS (10356 req/s), DGC-SHM and DGC-RDMA reduce P99 latency by 64.4% and 60.3%, respectively.

Additionally, it can be seen on the right side of the figure that DGC-SHM and DGC-RDMA trend towards P99 divergence earlier than the baseline GC. This behavior is attributed to the fact that, in our experimental setup, the DGC runtime operates with two fewer cores compared to the baseline runtime. As a result, DGC’s performance degrades faster than the baseline under extremely high loads. This is the trade-off accepted by DGC to optimize application performance under moderate loads.

To identify the source of this performance improvement, we examined the phase-wise latency behavior of DGC under the same configuration used in Figure 1. As observed in

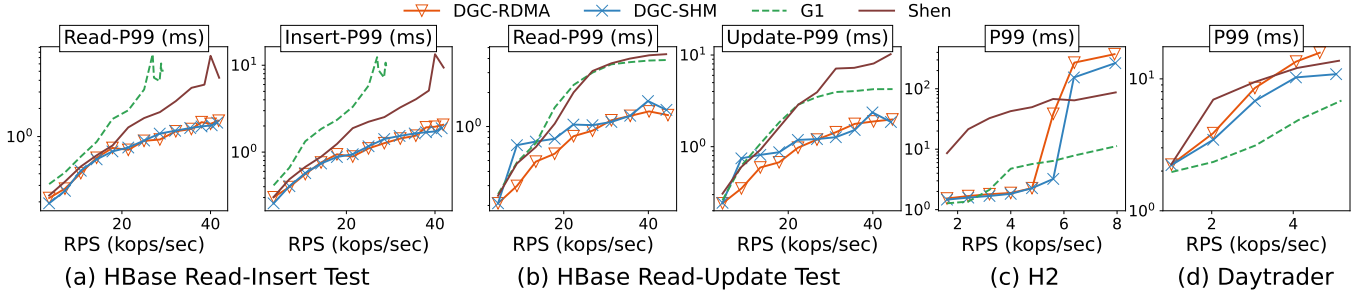


Figure 6: Throughput-latency curves of HBase, H2 and Daytrader

Figure 5, the performance fluctuations during the marking phase are significantly smoothed out with DGC. Residual latency fluctuations appear only during the compaction phase, which are attributed to Shenandoah’s load barriers.

HBase. We use HBase to further evaluate the performance of DGC in database scenarios. In this experiment, YCSB’s two mixed workloads (<50% read, 50% update> and <50% read, 50% insert>) [5] are running on a two-RegionServer configuration, while the RegionServers are equipped with different collectors to evaluate the improvement of DGC.

As shown in Figure 6 (a) and (b), DGC-SHM and DGC-RDMA both achieve lower P99 latency than baselines under most of the issue rates. For the <read, update> workload, DGC-SHM’s P99 read and update latencies are 63.9% and 56.2% lower than G1 under peak throughput, while DGC-RDMA’s are 67.5% and 53.3% lower than G1. (G1 is the the two baselines) For the <read, insert> workload, DGC-SHM’s P99 read and insert latency are 66.9% and 55.8% lower than Shenandoah under peak throughput, while DGC-RDMA’s P99 read and insert latency are 65.3% and 78.0% lower than Shenandoah. G1 fails to achieve a similar peak throughput as Shenandoah or DGC, mainly due to frequent STW full GC (a fallback mechanism triggers when the concurrent marking in mixed GC fails to meet the allocation rate). In contrast, although Shenandoah induces shorter and fewer stop-the-world pauses, its concurrent GC phases cause a consistently slower request processing speed. This affects a much larger number of requests and leads to worse P99 latency compared to G1. DGC not only maintains the low-pause advantage of Shenandoah but also resolves the CPU contention issue during concurrent GC. As a result, it demonstrates significantly improved tail latency.

H2. As shown in Figure 6 (c), DGC’s evaluation results on H2 is similar to that in HBase. The tail latency of DGC remains lower than Shenandoah and G1 under moderate throughput (between 3.2k and 4.8k ops/s). When the throughput is 4k ops/s, DGC-SHM’s P99 latency is 61.7% lower than G1, while DGC-RDMA’s P99 latency is 60.3% lower than G1 (G1 is the better one among the two baselines in H2). Unlike HBase, the P99 latency of DGC increases in H2 when throughput further

increases. The reason is the same as Specjbb2015: due to the static CPU resource partition, the served JVMs have fewer cores and thus perform worse when under high throughput. G1 performs the best under those configurations thanks to its efficient STW young generation GC.

DayTrader. Figure 6 (d) shows the performance evaluation of DGC on DayTrader. Under moderate workloads, DGC-SHM and DGC-RDMA optimize the P99 latency by up to 50.4% and 44.3% compared to Shenandoah. However, unlike the previous benchmarks, G1 consistently outperforms all other collectors in this test. Analysis of the GC logs reveals that this is due to G1’s generational design. In DayTrader, G1’s average young generation GC pause is merely 2ms. This indicates that object lifecycles in this workload are extremely ephemeral, resulting in young generation scans encountering almost no live objects. Since our implementation is based on OpenJDK 17, where Shenandoah lacks generational support, DGC is unable to benefit from this specific characteristic. Nevertheless, the substantial performance advantage of DGC over Shenandoah still highlights the superiority of the DGC system design.

8.3 RDMA and Memory Efficiency

Figure 7 illustrates the SPECjbb2015 tail latency and corresponding RDMA traffic of DGC-RDMA with varying local heap cache sizes. To eliminate the noise caused by drastic load fluctuations in the HBIR mode, we configured SPECjbb2015 to the fixed-throughput (Preset) mode. The Injection Rate (IR) was fixed at 10,356, corresponding to the best Critical jOPS of the baseline Shenandoah collector (as shown in Figure 4), to explicitly demonstrate the performance uplift enabled by DGC. Additionally, the coordinator enforces strict time-multiplexing of the four GC cores between two Runtimes to prevent simultaneous DGC triggers and avoid doubling peak heap cache usage. Among the four evaluated configurations, even the most constrained setting (1/4 cache) achieves a 51.5% improvement in P99 latency over Shenandoah, since its RDMA traffic is still within the bandwidth limit. Notably, the 1/2 cache configuration strikes an optimal balance between memory overhead and RDMA traffic cost. Compared to the

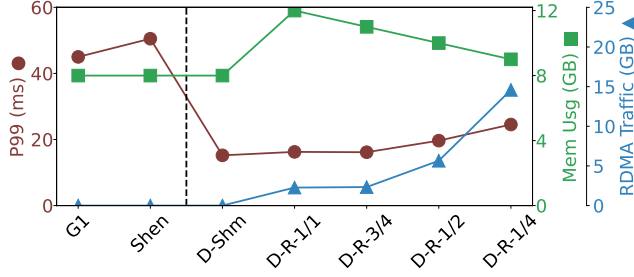


Figure 7: DGC-RDMA’s P99 latency, memory usage, and RDMA traffic under different local cache settings

baseline of two 4GB heaps, it requires only 2GB of additional remote memory (a 25% overhead) to serve as the DGC node’s local cache. Furthermore, it incurs an average RDMA traffic of only 5.52GB per GC cycle, representing a moderate 37% amplification relative to the 4GB full heap size.

8.4 Scalability Test

SPECjbb2015 homogeneous workload. To evaluate the performance of DGC as the number of runtimes increases, we conducted a series of experiments running SPECjbb2015 with varying numbers of backends in a distributed setup. The experiment adhered to the core allocation policy established earlier. To simplify the testing process, we used SPECjbb2015’s PRESET mode, in which the benchmark runs at a fixed throughput and reports P99 latency. The average load per backend was set to 6819 req/s, as measured in Table 1, and the total system load was scaled proportionally with the number of backends.

The results are shown in Figure 8. DGC consistently demonstrated better P99 latency than Shenandoah across all scales, with the improvement of up to 43.3% and 50.9% at 6 backends. Since DGC-SHM requires the served JVM and the disaggregated marker to reside on the same node, it was limited to 6 backends (7 backends would require at least 70 cores per node, exceeding the 64 cores limit in our physical machines). DGC-RDMA mode was tested up to 12 backends, but the performance improvement began to diminish. Analysis of the GC logs indicates that the RDMA NIC bandwidth became a bottleneck at this point.

During the test, we also recorded the time taken by the CP-SAT solving process. The solving time remained efficient, taking only 11.1 ms even in the 12-backend scenario, which is sufficient for scheduling the GC tasks that last for hundreds of milliseconds.

Multi-application heterogeneous workload. To simulate a realistic scenario with multiple applications colocating on a single physical machine, we constructed a mixed load including instances from different applications. In this test, we configured four JVMs on one machine: two running SPECjbb2015 backends under PRESET mode and two running HBase region-

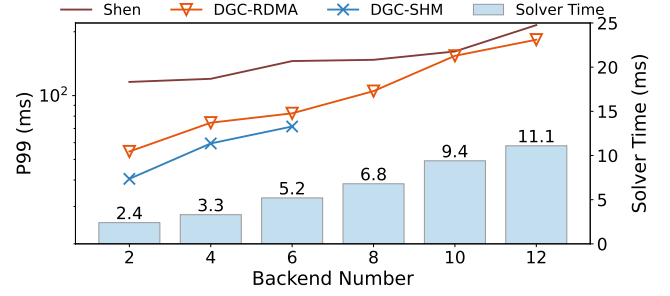


Figure 8: Scalability test results on SPECjbb2015 PRESET

Table 3: Performance of DGC-SHM and DGC-RDMA on multi-application mixed workload

	P99 (ms)	SPECjbb2015	HBase Read	HBase Update
DGC-SHM	42		1.56	1.85
DGC-RDMA	61		1.89	2.89
Shenandoah	116		4.37	10.21

Server instances under YCSB’s read-update mixed workloads. This setup reflects a common enterprise deployment where online services and database workloads represent two major types of production loads. In the DGC configuration, each runtime was allocated 8 cores, while the disaggregated part was assigned 8 cores (2 per runtime $\times$ 4 served runtimes). For the baseline Shenandoah configuration, each runtime was allocated 10 cores and performed garbage collection locally.

The performance results are shown in Table 3. When evaluated using the geometric mean of the three latency metrics, DGC-SHM reduced P99 latency by 71.4% compared to the baseline, while DGC-RDMA achieved a reduction of 60.0%.

8.5 Ablation Study

We evaluated the individual contributions of our two primary optimizations using SPECjbb2015’s PRESET mode. Given the limited configurability of DGC-SHM, this ablation study focuses on DGC-RDMA, with results detailed in Table 4.

Impact of Pipelined Co-design. The "noOPT" configuration disables the pipelined marker-transfer co-design described in Section 5.2. Instead of concurrent execution, the marker is forced to wait for the entire heap to be copied before starting. This serialization increases the average marking time by 134 ms—accounting for 83.8% of the theoretical 160 ms required to transfer a 4 GB heap. This gap confirms that our design effectively masks the majority of data transfer costs.

Impact of Global Scheduling. The "noCOOR" configuration removes the global GC scheduler, relying instead on local JVM heuristics. This lack of coordination leads to probabilistic overlapping of GC tasks, increasing the variability of marking times and degrading end-to-end P99 latency.

Overall Performance. When both optimizations are disabled, P99 latency regresses to 107 ms, closely mirroring the native Shenandoah baseline (116 ms). Conversely, with both enabled, the 4-thread DGC-RDMA marker completes the task in 506 ms, incurring only a marginal 12.9% overhead compared to a local 4-thread Shenandoah marker.

Table 4: Performance breakdown on SPECjbb2015 PRESET

Garbage Collector	P99 (ms)	avg. Mark Time (ms)
DGC	49	506
DGC-noOPT	68	640
DGC-noCOOR	79	593
DGC-noOPT-noCOOR	107	774
Shenandoah	116	448

9 Related Work

9.1 Concurrent garbage collection

Concurrent garbage collection (GC) is a critical component in modern Java Virtual Machines (JVMs) to ensure efficient memory management without significant pause times. Several state-of-the-art concurrent marking algorithms have been developed, including those used in the JDK’s G1, Shenandoah, and ZGC collectors.

Other research works also focus on optimizing the concurrent marking GC algorithm. C4 [40] builds on Pauseless GC by incorporating a generational approach, while Collie [18] suggests leveraging hardware transactional memory (HTM) for atomic object relocation. Compressor [22] employs hand-over-hand compaction to maintain minimal physical memory usage. Block-free GC [35] introduces non-blocking handshakes for concurrent stack scanning and object copying. Jade [50] provides a group-wise collection mechanism to improve the performance of low-pause GC. Some other concurrent collectors, like LXR [55], use reference counting to discover dead objects but still need to conduct concurrent marking periodically. These works include concurrent marking phases and can potentially benefit from our work.

9.2 Externalized Garbage Collection

Due to the increasing demand for high-performance computing and data processing, the use of specialized hardware to accelerate applications has become increasingly popular [20, 21, 23, 31, 42, 44, 48, 51, 52, 54, 57].

Previous research has also explored the idea of externalizing GC processes with specialized hardware. Martin Maas et al. [30] proposed a hardware accelerator that is located close to the memory controller, which can significantly improve the GC performance. Andrés Amaya et al. [9] further extended the idea in real-time scenarios. Thomas et al. [41] proposed to optimize

the garbage collection with recently developed near-memory processing (NMP) hardware. DGC is orthogonal to these works, since DGC uses server CPU to offload the garbage collection and relies on RDMA to transfer data between runtime and collector, which leads to different challenges. Also, unlike these works that rely on exclusive access to garbage collection resources, DGC introduces the capability to share GC resources across multiple runtimes, improving overall resource utilization.

9.3 Disaggregation and Resource Pooling

Resource pooling is a widely adopted approach for improving resource utilization in computing systems. Multiple existing studies have focused specifically on the problem of memory pooling [1, 4, 13, 15, 24, 25, 29, 38, 39, 47, 56].

Some works also focus on building a garbage collector for memory-disaggregated systems. Semeru [45] and Mako [28] are memory-disaggregated managed runtimes with a distributed garbage collector that is located on the memory node to minimize the memory swapping overhead. MemLiner [46] further reduces the memory swapping overhead by locating the garbage collector on the CPU node and prioritizing the marking task of objects in the CPU Node’s local memory. These efforts primarily focus on leveraging disaggregated memory to address memory utilization issues. Their goals are orthogonal to those of DGC, which is chiefly concerned with mitigating CPU load fluctuations during garbage collection and improving CPU utilization through decoupled GC execution.

10 Conclusion

This work provides DGC, a disaggregated architecture to serve GC tasks of language runtimes in an external instance. DGC supports externalizing the costly marking task in GC to idle CPU cores locally or remotely and allowing multiple runtimes to share the GC service with minimized conflicts. The evaluation results on representative latency-sensitive applications show that DGC can significantly improve the application latency and goodput.

References

- [1] Emmanuel Amaro, Christopher Branner-Augmon, Zhihong Luo, Amy Ousterhout, Marcos K Aguilera, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. 2020. Can far memory improve job throughput?. In *Proceedings of the Fifteenth European Conference on Computer Systems*. 1–16.
- [2] Apache. 2022. Apache HBase. <https://hbase.apache.org/>.
- [3] Stephen M Blackburn, Zixian Cai, Rui Chen, Xi Yang, John Zhang, and John Zigman. 2025. Rethinking Java Performance Analysis. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume*

- 1, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025. ACM. doi:10.1145/3669940.3707217
- [4] Lei Chen, Shi Liu, Chenxi Wang, Haoran Ma, Yifan Qiao, Zhe Wang, Chenggang Wu, Youyou Lu, Xiaobing Feng, Huimin Cui, et al. 2024. A Tale of Two Paths: Toward a Hybrid Data Plane for Efficient {Far-Memory} Applications. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 77–95.
- [5] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM symposium on Cloud computing*. 143–154.
- [6] Standard Performance Evaluation Corporation. 2021. The specjbb2015 benchmark. <https://www.spec.org/jbb2015/>.
- [7] David Detlefs, Christine H. Flood, Steve Heller, and Tony Printezis. 2004. Garbage-first garbage collection. In *Proceedings of the 4th International Symposium on Memory Management, ISMM 2004, Vancouver, BC, Canada, October 24-25, 2004*. ACM, 37–48. doi:10.1145/1029873.1029879
- [8] Christine H Flood, Roman Kennke, Andrew Dinn, Andrew Haley, and Roland Westrelin. 2016. Shenandoah: An open-source concurrent compacting garbage collector for openjdk. In *Proceedings of the 13th International Conference on Principles and Practices of Programming on the Java Platform: Virtual Machines, Languages, and Tools*. 1–9.
- [9] Andrés Amaya García, David May, and Ed Nutting. 2021. Integrated hardware garbage collection. *ACM Transactions on Embedded Computing Systems (TECS)* 20, 5 (2021), 1–25.
- [10] Google. 2025. Android Open Source Project. <https://source.android.com/docs/core>.
- [11] Google. 2025. The Go Programming Language. <https://go.dev>.
- [12] Google. 2025. V8 JavaScript engine. <https://v8.dev>.
- [13] Juncheng Gu, Youngmoon Lee, Yiwen Zhang, Mosharaf Chowdhury, and Kang G Shin. 2017. Efficient memory disaggregation with infiniswap. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 649–667.
- [14] Jing Guo, Zihao Chang, Sa Wang, Haiyang Ding, Yihui Feng, Liang Mao, and Yungang Bao. 2019. Who limits the resource efficiency of my datacenter: An analysis of alibaba datacenter traces. In *Proceedings of the international symposium on quality of service*. 1–10.
- [15] Zhiyuan Guo, Zijian He, and Yiyang Zhang. 2023. Mira: A program-behavior-guided far memory system. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 692–708.
- [16] H2. 2022. H2 database engine. <https://www.h2database.com/html/main.html>.
- [17] IBM. 2023. IBM Daytrader. <https://www.ibm.com/docs/en/linux-on-systems?topic=bad-daytrader>.
- [18] Balaji Iyengar, Gil Tene, Michael Wolf, and Edward Gehringer. 2012. The collie: a wait-free compacting collector. In *Proceedings of the 2012 international symposium on Memory Management*. 85–96.
- [19] Nicholas Jacek and J Eliot B Moss. 2019. Learning when to garbage collect with random forests. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on Memory Management*. 53–63.
- [20] Houxiang Ji, Mark Mansi, Yan Sun, Yifan Yuan, Jinghan Huang, Reese Kuper, Michael M Swift, and Nam Sung Kim. 2023. {STYX}: Exploiting {SmartNIC} capability to reduce datacenter memory tax. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 619–633.
- [21] Xin Jin, Xiaozhou Li, Haoyu Zhang, Robert Soulé, Jeongkeun Lee, Nate Foster, Changhoon Kim, and Ion Stoica. 2017. Netcache: Balancing key-value stores with fast in-network caching. In *Proceedings of the 26th symposium on operating systems principles*. 121–136.
- [22] Haim Kermany and Erez Petrank. 2006. The Compressor: concurrent, incremental, and parallel compaction. In *Proceedings of the 27th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 354–363.
- [23] Jongyul Kim, Insu Jang, Waleed Reda, Jaeseong Im, Marco Canini, Dejan Kostić, Youngjin Kwon, Simon Peter, and Emmett Witchel. 2021. Linefs: Efficient smartnic offload of a distributed file system with pipeline parallelism. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 756–771.
- [24] Andres Lagar-Cavilla, Junwhan Ahn, Suleiman Souhlal, Neha Agarwal, Radoslaw Burny, Shakeel Butt, Jichuan Chang, Ashwin Chaugule, Nan Deng, Junaid Shahid, et al. 2019. Software-defined far memory in warehouse-scale computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 317–330.
- [25] Quanxi Li, Hong Huang, Ying Liu, Yanwen Xia, Jie Zhang, Mosong Zhou, Xiaobing Feng, Huimin Cui, Quan Chen, Yizhou Shan, et al. 2025. Beehive: A Scalable Disaggregated Memory Runtime Exploiting Asynchrony of Multithreaded Programs. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 167–187.
- [26] Heng Liao, Bingyang Liu, Xianping Chen, Zhigang Guo, Chuanning Cheng, Jianbing Wang, Xiangyu Chen, Peng Dong, Rui Meng, Wenjie Liu, et al. 2025. UB-Mesh: a Hierarchically Localized nD-FullMesh Datacenter Network Architecture. *arXiv preprint arXiv:2503.20377* (2025).
- [27] Compute Express Link. 2025. Compute Express Link (CXL) Specification 4.0. https://computeexpresslink.org/wp-content/uploads/2025/11/CXL_4.0-Specification-Release_FINAL_Website-Copy.pdf.
- [28] Haoran Ma, Shi Liu, Chenxi Wang, Yifan Qiao, Michael D Bond, Stephen M Blackburn, Miryung Kim, and Guoqing Harry Xu. 2022. Mako: A low-pause, high-throughput evacuating collector for memory-disaggregated datacenters. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 92–107.
- [29] Haoran Ma, Yifan Qiao, Shi Liu, Shan Yu, Yuanjiang Ni, Qingda Lu, Jiesheng Wu, Yiyang Zhang, Miryung Kim, and Harry Xu. 2024. {DRust}:{Language-Guided} Distributed Shared Memory with Fine Granularity, Full Transparency, and Ultra Efficiency. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 97–115.
- [30] Martin Maas, Krste Asanović, and John Kubiawicz. 2018. A hardware accelerator for tracing garbage collection. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 138–151.
- [31] Young Gyoung Moon, Ilwoo Park, Seungeon Lee, and Kyoung Soo Park. 2018. Accelerating flow processing middleboxes with programmable NICs. In *Proceedings of the 9th Asia-Pacific Workshop on Systems*. 1–3.
- [32] New Relic. 2024. 2024 State of the Java Ecosystem. Technical Report. New Relic. <https://newrelic.com/sites/default/files/2024-04/new-relic-state-of-the-java-ecosystem-report-2024-04-30.pdf> Accessed: 2024-05-20.

- [33] OpenJDK. 2018. JEP 310: Application Class-Data Sharing. <https://openjdk.java.net/jeps/310>
- [34] OpenJDK. 2025. OpenJDK HotSpot. <https://openjdk.org/groups/hotspot/>.
- [35] Erik Österlund and Welf Löwe. 2016. Block-free concurrent GC: stack scanning and copying. In *Proceedings of the 2016 ACM SIGPLAN International Symposium on Memory Management*. 1–12.
- [36] Laurent Perron and Frédéric Didier. 2024. CP-SAT. Google. https://developers.google.com/optimization/cp/cp_solver/
- [37] Laurent Perron and Vincent Furnon. 2024. OR-Tools. Google. <https://developers.google.com/optimization/>
- [38] Yifan Qiao, Chenxi Wang, Zhenyuan Ruan, Adam Belay, Qingda Lu, Yiyang Zhang, Miryung Kim, and Guoqing Harry Xu. 2023. Hermit: {Low-Latency}, {High-Throughput}, and Transparent Remote Memory via {Feedback-Directed} Asynchrony. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 181–198.
- [39] Zhenyuan Ruan, Malte Schwarzkopf, Marcos K Aguilera, and Adam Belay. 2020. {AIFM}: {High-Performance}, {Application-Integrated} far memory. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 315–332.
- [40] Gil Tene, Balaji Iyengar, and Michael Wolf. 2011. C4: The continuously concurrent compacting collector. In *Proceedings of the international symposium on memory management*. 79–88.
- [41] Samuel Thomas, Jiwon Choe, Ofir Gordon, Erez Petrank, Tali Moreshet, Maurice Herlihy, and R Iris Bahar. 2022. Towards Hardware Accelerated Garbage Collection with Near-Memory Processing. In *2022 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–6.
- [42] Maroun Tork, Lina Maudlej, and Mark Silberstein. 2020. Lynx: A smartnic-driven accelerator-centric architecture for network servers. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 117–131.
- [43] TPC. 2025. TPC-C. <https://www.tpc.org/tpcc/>.
- [44] Lluís Vilanova, Lina Maudlej, Shai Bergman, Till Miemietz, Matthias Hille, Nils Asmussen, Michael Roitzsch, Hermann Härtig, and Mark Silberstein. 2022. Slashing the disaggregation tax in heterogeneous data centers with fractos. In *Proceedings of the seventeenth european conference on computer systems*. 352–367.
- [45] Chenxi Wang, Haoran Ma, Shi Liu, Yuanqi Li, Zhenyuan Ruan, Khanh Nguyen, Michael D Bond, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. 2020. Semeru: A {Memory-Disaggregated} managed runtime. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 261–280.
- [46] Chenxi Wang, Haoran Ma, Shi Liu, Yifan Qiao, Jonathan Eyolfson, Christian Navasca, Shan Lu, and Guoqing Harry Xu. 2022. {MemLiner}: Lining up Tracing and Application for a {Far-Memory-Friendly} Runtime. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 35–53.
- [47] Chenxi Wang, Yifan Qiao, Haoran Ma, Shi Liu, Wenguang Chen, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. 2023. Canvas: Isolated and adaptive swapping for {Multi-Applications} on remote memory. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 161–179.
- [48] Xingda Wei, Rongxin Cheng, Yuhang Yang, Rong Chen, and Haibo Chen. 2023. Characterizing off-path {SmartNIC} for accelerating distributed systems. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 987–1004.
- [49] Mingyu Wu. 2024. H2 Throttle. <https://github.com/SJTU-IPADS/jade-artifacts/tree/master/h2-throttle>.
- [50] Mingyu Wu, Liang Mao, Yude Lin, Yifeng Jin, Zhe Li, Hongtao Lyu, Jiawei Tang, Xiaowei Lu, Hao Tang, Denghui Dong, et al. 2024. Jade: A High-throughput Concurrent Copying Garbage Collector. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 1160–1174.
- [51] Tong Xing, Hesam Tajbakhsh, Israat Haque, Michio Honda, and Antonio Barbalace. 2022. Towards portable end-to-end network performance characterization of smartnics. In *Proceedings of the 13th ACM SIGOPS Asia-Pacific Workshop on Systems*. 46–52.
- [52] Zhaoqi Xiong and Noa Zilberman. 2019. Do switches dream of machine learning? toward in-network classification. In *Proceedings of the 18th ACM workshop on hot topics in networks*. 25–33.
- [53] Albert Mingkun Yang and Tobias Wrigstad. 2022. Deep dive into zgc: A modern garbage collector in openjdk. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 44, 4 (2022), 1–34.
- [54] Zhuolong Yu, Yiwen Zhang, Vladimir Braverman, Mosharaf Chowdhury, and Xin Jin. 2020. Netlock: Fast, centralized lock management using programmable switches. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. 126–138.
- [55] Wenyu Zhao, Stephen M Blackburn, and Kathryn S McKinley. 2022. Low-latency, high-throughput garbage collection. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 76–91.
- [56] Yang Zhou, Hassan MG Wassel, Sihang Liu, Jiaqi Gao, James Mickens, Minlan Yu, Chris Kennelly, Paul Turner, David E Culler, Henry M Levy, et al. 2022. Carbink: {Fault-Tolerant} far memory. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 55–71.
- [57] Hang Zhu, Kostis Kaffes, Zixu Chen, Zhenming Liu, Christos Kozyrakis, Ion Stoica, and Xin Jin. 2020. {RackSched}: A {Microsecond-Scale} scheduler for {Rack-Scale} computers. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 1225–1240.